Department of Computer Science and Engineering
University of Moratuwa

CS4533 – Concurrent Programming

Lab 06: Solution Patterns for Parallel Programming

Module:	CS4533 – Concurrent Programming
Lab:	Lab 06 – Solution Patterns for Parallel Programming
Duration:	2 hours (in-class)
Topics:	Loop parallel, divide & conquer, pipeline, task queue, work stealing

Prerequisites

- Completed Labs 01–05
- Compile with: gcc -pthread -fopenmp -O2 -o output source.c (some exercises need OpenMP)
- Familiarity with pthreads, OpenMP, mutexes, condition variables, and bounded queues

Learning Objectives

By the end of this lab, you should be able to:

- Apply the right parallel pattern to a given problem
- Implement pipeline parallelism with bounded queues between stages
- Build a thread pool with a shared task queue for dynamic load balancing
- Implement work stealing for unbalanced workloads
- Reason about when to use each pattern based on problem characteristics

Exercise 1: Loop Parallel — Mandelbrot Set (0.5 marks)

Background

The Mandelbrot set is computed per-pixel: for each point c in the complex plane, iterate $z = z^2 + c$ until $|z| > 2$ or a maximum iteration count is reached. Crucially, different pixels require vastly different computation — points inside the set hit max iterations while points outside escape quickly. This makes it an ideal test for scheduling policies.

Recall: Static vs Dynamic Scheduling

In Lab 3, you experimented with OpenMP scheduling on a uniform workload (blur filter). Here is a quick reminder:

- Static: iterations are divided equally among threads upfront (e.g., Thread 0 gets rows 0–24, Thread 1 gets 25–49, etc.). No runtime overhead, but if some rows are heavier than others, fast threads sit idle.

- Dynamic: threads pull iterations from a shared pool one at a time as they become free. Small overhead per iteration, but no thread idles while work remains.

For the blur filter (uniform cost per row), static was fastest. The Mandelbrot set has highly non-uniform cost — rows near the centre are much heavier. This exercise shows when dynamic scheduling becomes essential.

Provided Code

The following serial code computes the Mandelbrot set and writes a PGM image. The benchmark is included.

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define W 2000
#define H 2000
#define MAX_ITER 1000

int image[H][W];

int mandelbrot(double cx, double cy) {
    double zx = 0, zy = 0;
    int iter = 0;
    while (zx*zx + zy*zy <= 4.0 && iter < MAX_ITER) {
        double tmp = zx*zx - zy*zy + cx;
        zy = 2.0*zx*zy + cy;
        zx = tmp;
        iter++;
    }
    return iter;
}

void compute_serial() {
    for (int j = 0; j < H; j++)
        for (int i = 0; i < W; i++) {
            double cx = -2.0 + 3.0*i/W;
            double cy = -1.5 + 3.0*j/H;
            image[j][i] = mandelbrot(cx, cy);
        }
}

void save_pgm(const char *fname) {
    FILE *f = fopen(fname, "w");
    fprintf(f, "P2\n%d %d\n%d\n", W, H, MAX_ITER);
    for (int j = 0; j < H; j++)
        for (int i = 0; i < W; i++)
            fprintf(f, "%d\n", image[j][i]);
    fclose(f);
}

int main() {
    double t;

    t = omp_get_wtime();
    compute_serial();
```

```
    printf("Serial: %.4fs\n", omp_get_wtime()-t);
    save_pgm("mandelbrot.pgm");

    // TODO: Call compute_static and compute_dynamic
    // with 1,2,4,8 threads and print times
    return 0;
}

// ======================================
// TODO: Implement these two functions
// ======================================

// Same logic as compute_serial(), but add:
// #pragma omp parallel for schedule(static)
// on the OUTER loop (j loop)
void compute_static(int num_threads) {
    omp_set_num_threads(num_threads);
    // TODO: Copy the nested loop from compute_serial
    // and add the OpenMP pragma with static scheduling
}

// Same logic as compute_serial(), but add:
// #pragma omp parallel for schedule(dynamic)
// on the OUTER loop (j loop)
void compute_dynamic(int num_threads) {
    omp_set_num_threads(num_threads);
    // TODO: Copy the nested loop from compute_serial
    // and add the OpenMP pragma with dynamic scheduling
}
```

Your Task

- Write compute_static(): parallelize the outer loop with #pragma omp parallel for schedule(static)
- Write compute_dynamic(): same but with schedule(dynamic)
- Run both with 1, 2, 4, and 8 threads. Verify images match the serial version.

Questions

Q1.1: Present a table comparing static vs dynamic scheduling across thread counts. Which is faster and by how much? Why does the difference exist for the Mandelbrot set but not for the blur filter in Lab 3?

Exercise 2: Divide & Conquer — Parallel Tree Reduction

Background

Divide & conquer works by recursively splitting a problem, solving subproblems in parallel, and combining results. Unlike Lab 3's merge sort (balanced split), this exercise uses an unbalanced binary tree where subtree sizes vary greatly — making granularity control more important.

Provided Code

```
#include <stdio.h>
```

```
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

typedef struct node {
    int value;
    struct node *left, *right;
} node_t;

// Build an unbalanced tree: left-heavy
node_t *build_tree(int depth, int max_depth, int *id) {
    if (depth > max_depth) return NULL;
    node_t *n = malloc(sizeof(node_t));
    n->value = ++(*id);
    // Left subtree goes deeper than right
    n->left  = build_tree(depth+1, max_depth, id);
    n->right = build_tree(depth+1,
        max_depth - (depth%2 == 0 ? 2 : 0), id);
    return n;
}

long serial_sum(node_t *n) {
    if (!n) return 0;
    return n->value
         + serial_sum(n->left)
         + serial_sum(n->right);
}

int count_nodes(node_t *n) {
    if (!n) return 0;
    return 1 + count_nodes(n->left)
           + count_nodes(n->right);
}

int main() {
    int id = 0;
    node_t *root = build_tree(0, 22, &id);
    printf("Nodes: %d\n", count_nodes(root));

    struct timespec s, e;
    clock_gettime(CLOCK_MONOTONIC, &s);
    long sum_s = serial_sum(root);
    clock_gettime(CLOCK_MONOTONIC, &e);
    double ts = (e.tv_sec-s.tv_sec)
              +(e.tv_nsec-s.tv_nsec)/1e9;
    printf("Serial sum: %ld (%.4fs)\n", sum_s, ts);

    // TODO: Implement parallel_sum using pthreads
    // Fork for left/right subtrees above a
    // size threshold; serial below threshold.
    // Verify result matches serial_sum.

    return 0;
}
```

Your Task

- Write parallel_sum() that spawns a new thread for each subtree above a size threshold (e.g., 1000 nodes). Below the threshold, use serial_sum().
- Experiment with different thresholds: 100, 1000, 10000. Report which is fastest.
- Verify the parallel result matches the serial result.

Questions

Q2.1: Present timing results for different thresholds. Why does a very low threshold (100) hurt performance? Why does a very high threshold (10000) also underperform compared to the optimal?

Exercise 3: Pipeline Pattern

Background

A pipeline breaks processing into sequential stages, each running in its own thread. Data flows from stage to stage through bounded queues. Individual items don't get faster, but throughput increases because all stages work concurrently on different items — like an assembly line.

Provided Code

A bounded queue (from Lab 5) and the benchmark harness are provided. You implement the 3-stage pipeline logic.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <math.h>
#include <time.h>

// --- Simple bounded queue (provided) ---
typedef struct qnode { int val; struct qnode *next; } qn_t;
typedef struct {
    qn_t *head, *tail;
    pthread_mutex_t lock;
    pthread_cond_t not_full, not_empty;
    int count, cap;
} queue_t;

void q_init(queue_t *q, int cap) {
    qn_t *s = malloc(sizeof(qn_t));
    s->val=-1; s->next=NULL;
    q->head=s; q->tail=s; q->count=0; q->cap=cap;
    pthread_mutex_init(&q->lock,NULL);
    pthread_cond_init(&q->not_full,NULL);
    pthread_cond_init(&q->not_empty,NULL);
}
void q_enq(queue_t *q, int v) {
    qn_t *n=malloc(sizeof(qn_t));
    n->val=v; n->next=NULL;
    pthread_mutex_lock(&q->lock);
    while(q->count>=q->cap)
        pthread_cond_wait(&q->not_full,&q->lock);
```

```
    q->tail->next=n; q->tail=n; q->count++;
    pthread_cond_signal(&q->not_empty);
    pthread_mutex_unlock(&q->lock);
}
int q_deq(queue_t *q) {
    pthread_mutex_lock(&q->lock);
    while(q->count==0)
        pthread_cond_wait(&q->not_empty,&q->lock);
    qn_t *old=q->head; q->head=old->next;
    int v=q->head->val; q->count--;
    free(old);
    pthread_cond_signal(&q->not_full);
    pthread_mutex_unlock(&q->lock);
    return v;
}

#define NUM_ITEMS 5000
#define POISON -999

// --- Stage functions (simulated work) ---
// Stage 1: Generate raw data
int stage1_generate(int id) {
    return id * 3 + 7;  // simple transform
}

// Stage 2: Heavy processing (bottleneck)
int stage2_process(int data) {
    double result = data;
    for (int i = 0; i < 5000; i++)
        result = sin(result) * cos(result) + 1.0;
    return (int)result + data;
}

// Stage 3: Finalize
int stage3_finalize(int data) {
    return data % 10000;
}

// --- Serial baseline (provided) ---
void run_serial() {
    long checksum = 0;
    for (int i = 0; i < NUM_ITEMS; i++) {
        int d = stage1_generate(i);
        d = stage2_process(d);
        d = stage3_finalize(d);
        checksum += d;
    }
    printf("Serial checksum: %ld\n", checksum);
}
```

Your Task

- Create two bounded queues: q12 (between stage 1 and 2) and q23 (between stage 2 and 3).
- Write three thread functions:

- stage1_thread: generates items 0..NUM_ITEMS-1 using stage1_generate(), enqueues to q12, then sends a POISON pill.
- stage2_thread: dequeues from q12, processes with stage2_process(), enqueues to q23. Stops on POISON (forwards it).
- stage3_thread: dequeues from q23, finalizes with stage3_finalize(), accumulates a checksum. Stops on POISON.

- Launch all three threads, join, print the checksum. Verify it matches the serial version.
- Then modify: run 2 or 3 instances of stage2_thread (the bottleneck) to see throughput improvement.

Questions

Q3.1: Compare serial time vs pipeline time with 1 stage-2 thread, then with 2 and 3 stage-2 threads. Present a table. Why does adding more stage-2 threads help?

Q3.2: What determines the maximum throughput of a pipeline? If stage 2 takes 10ms per item and stages 1 and 3 take 1ms each, what is the theoretical throughput with 1 vs 3 stage-2 threads?

Exercise 4: Task Queue with Thread Pool

Background

A thread pool creates a fixed number of worker threads that pull tasks from a shared queue. This avoids the overhead of creating/destroying threads per task and provides dynamic load balancing — fast workers automatically take more tasks.

Provided Code

The task structure, task generation, and verification are provided. You implement the thread pool.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <math.h>
#include <time.h>

#define NUM_TASKS 200

typedef struct {
    int id;
    int difficulty;  // iterations of heavy work
    double result;
} task_t;

task_t tasks[NUM_TASKS];

void do_task(task_t *t) {
    double val = t->id;
    for (int i = 0; i < t->difficulty; i++)
        val = sin(val) * cos(val) + 1.0;
    t->result = val;
}
```

```
void generate_tasks() {
    srand(42);
    for (int i = 0; i < NUM_TASKS; i++) {
        tasks[i].id = i;
        // Uneven difficulty: some tasks 100x harder
        tasks[i].difficulty = (rand()%10 == 0)
            ? 500000 : 5000;
        tasks[i].result = 0;
    }
}

void run_serial() {
    for (int i = 0; i < NUM_TASKS; i++)
        do_task(&tasks[i]);
}

// --- Thread pool (TODO) ---
// Shared task index (atomic or mutex-protected)
int next_task = 0;
pthread_mutex_t task_mutex = PTHREAD_MUTEX_INITIALIZER;

// TODO: Write pool_worker() — loop: grab next task
// index, do it, repeat until no tasks left.

int main() {
    generate_tasks();
    struct timespec s, e;

    clock_gettime(CLOCK_MONOTONIC, &s);
    run_serial();
    clock_gettime(CLOCK_MONOTONIC, &e);
    double ts = (e.tv_sec-s.tv_sec)
              +(e.tv_nsec-s.tv_nsec)/1e9;
    printf("Serial: %.4fs\n", ts);

    int thread_counts[] = {2, 4, 8};
    for (int t = 0; t < 3; t++) {
        int n = thread_counts[t];
        generate_tasks();  // reset
        next_task = 0;
        pthread_t threads[n];

        clock_gettime(CLOCK_MONOTONIC, &s);
        for (int i = 0; i < n; i++)
            pthread_create(&threads[i],NULL,
                           pool_worker,NULL);
        for (int i = 0; i < n; i++)
            pthread_join(threads[i], NULL);
        clock_gettime(CLOCK_MONOTONIC, &e);

        double tp = (e.tv_sec-s.tv_sec)
                  +(e.tv_nsec-s.tv_nsec)/1e9;
        printf("Pool (%d threads): %.4fs (speedup %.2fx)\n",
               n, tp, ts/tp);
    }
```

```
    return 0;
}
```

Your Task

- Implement pool_worker(): in a loop, atomically grab the next task index (lock task_mutex, read and increment next_task, unlock). If index >= NUM_TASKS, exit. Otherwise, call do_task(&tasks[index]).
- Run and report speedup with 2, 4, and 8 threads.

Questions

Q4.1: Why does the thread pool handle the uneven task difficulty well, even though some tasks are 100x harder? How does this compare to static partitioning (each thread gets NUM_TASKS/n tasks)?

Exercise 5: Work Stealing

Background

Work stealing extends the task queue idea: instead of one shared queue, each thread has its own local deque (double-ended queue). Threads push and pop from their own deque (no contention). When a thread's deque is empty, it steals from another thread's deque. This reduces lock contention compared to a single shared queue.

Your Task

Implement a work-stealing scheduler for the same task set as Exercise 4.

- Create a per-thread deque (a simple array-based deque is fine — no need for lock-free).
- Initially distribute tasks round-robin across thread deques (thread i gets tasks i, i+n, i+2n, ...).
- Each worker thread: pop a task from its own deque, execute it. When its deque is empty, pick a random other thread and try to steal a task from the other end of that thread's deque.
- Protect each deque with its own mutex (fine-grained — one lock per deque, not one global lock).
- Run with 2, 4, and 8 threads. Compare performance against Exercise 4's shared-queue thread pool.

Deque Interface (suggested)

```
typedef struct {
    int items[NUM_TASKS];
    int front, back;  // push/pop at back, steal from front
    pthread_mutex_t lock;
} deque_t;

void deque_init(deque_t *d) {
    d->front = 0; d->back = 0;
    pthread_mutex_init(&d->lock, NULL);
}

void deque_push(deque_t *d, int task_id) {
    pthread_mutex_lock(&d->lock);
    d->items[d->back++] = task_id;
    pthread_mutex_unlock(&d->lock);
}
```

```
// Pop from own deque (back end)
int deque_pop(deque_t *d) {
    // TODO: return -1 if empty
}

// Steal from another thread's deque (front end)
int deque_steal(deque_t *d) {
    // TODO: return -1 if empty
}
```

Questions

Q5.1: Compare work-stealing vs shared-queue throughput with 2, 4, and 8 threads. Is work stealing faster? Under what conditions would the difference be more pronounced?

Q5.2: Why does a work-stealing thread pop from the back of its own deque but steal from the front of another thread's deque? What advantage does this give for cache locality?

Exercise 6: Pattern Selection Analysis

Task

For each of the following problems, identify the most suitable parallel pattern from the lecture (loop parallel, fork/join, divide & conquer, pipeline, producer/consumer, task queue, or work stealing). Justify your choice in 2–3 sentences.

Problem A: A web server receives HTTP requests. Each request requires: parsing the request, querying a database, and formatting the response. The database query is 10x slower than parsing or formatting.

Problem B: You have a 100GB log file. Each line is independent. You need to count how many lines contain the word "ERROR".

Problem C: A web crawler starts from a seed URL. It fetches a page, extracts all links, and then crawls each link. Different pages have vastly different numbers of outgoing links.

Problem D: You need to multiply two 10,000 × 10,000 matrices. Each element of the result can be computed independently.

Submission Guidelines

- Submit source files (.c), one per exercise (Exercises 1–5), named as: Lab06_<IndexNumber1>_<IndexNumber2>_Ex<N>.c
- For Exercise 6, include your answers in the PDF report.
- Submit one PDF report with benchmark output, performance tables, and answers to all questions.
- Compile with: gcc -pthread -fopenmp -O2 -lm -o output source.c
- Submit via Moodle before the deadline.
